



INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Matrices en Java

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

Las notas de 3 alumnos en cuatro parciales

100	90	95	100
60	55	70	50
40	70	45	35

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Indices

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35
	0	1	2	3

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Declaración de la variable

En el momento que se declara la variable se establece el nombre y el tipo de los elementos.

```
int[] [] np;
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

En Java los corchetes se utilizan para declarar un arreglo

Creación del arreglo de dos dimensiones

En el momento que se crea el arreglo se establece la cantidad de elementos.

```
np = new int [nFil] [nCol];
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
```

Crearemos matrices con el mismo número de elementos en cada fila.

Subindicación

```
np [0]
```

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35
	0	1	2	3

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

En Java la primera fila se accede con el índice 0

Subindicación

`np [0] [2]`

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35

0 1 2 3

`np [0] [2]` es un número entero

Expresiones

`np [0] [2] > 60`

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35

0 1 2 3

`np [0] [2]` puede aplicarse a cualquier operador con operandos enteros

Errores de ejecución

`np [0] [4]`

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35

0 1 2 3

ERROR!!
Alguna de las clases debe asumir la responsabilidad de validar los índices.

El atributo length

`np . length`

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35

0 1 2 3

El número de filas

El atributo length

`np [0] . length`

0	100	90	95	100
1	60	55	70	50
2	40	70	45	35

0 1 2 3

El número de componentes de la fila con índice 0

Diseño

NotasParciales <<atributos de clase>> minimo_aprobacion=60 <<atributos de instancia>> np [][] entero <<Constructores>> NotasParciales(nFil,NCol: entero) <<Comandos>> establecerNota(n,f,c:entero) <<Consultas>> obtenerNota(f,c:entero): entero cantFilas(): entero cantColumnas(): entero cantAprobados(): entero cantAprobadosFila(f:entero): entero cantAprobadosColumna(c:entero): entero hayTodosDesaprobados(): boolean dosConsecutivosDes(f:entero): boolean	• NotasParciales(nFil,nCol: entero) Requiere nFil y nCol mayor a 0 • establecerNota(n, f,c:entero) Modifica la nota que corresponde a la fila f y columna c, si n está en el rango 0-100. Requiere f,c válidos. • obtenerNota(f,c:entero): entero Requiere f,c válidos.
--	--

Diseño

```

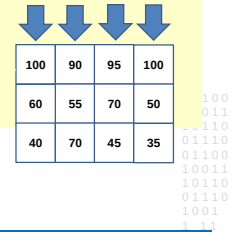
NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas():entero
cantAprobados():entero
cantAprobadosFila(f:entero):entero
cantAprobadosColumna(c:entero):entero
hayTodosDesaprobados():boolean
dosConsecutivosDes(f:entero):boolean
    
```

- **cantAprobados():entero**
Computa la cantidad de notas mayores al mínimo de aprobación.
- **cantAprobadosFila(f:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el alumno que corresponde a la fila f. Requiere f válida
- **cantAprobadosColumna(c:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el parcial que corresponde a la columna c. Requiere c válida.
- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Patrones de algoritmos - Contar

```

Algoritmo contar
contador ← 0
para cada fila f
    para cada columna c
        si np,c verifica la propiedad
            contador ← contador + 1
    
```



100	90	95	100
60	55	70	50
40	70	45	35

Contar la cantidad de elementos que verifican una propiedad es un recorrido exhaustivo

Patrones de algoritmos - Contar

```

Algoritmo contar los parciales aprobados
contador ← 0
para cada fila f
    para cada columna c
        si np,c > minimo_aprobacion
            contador ← contador + 1
    
```

Cada algoritmo puede refinarse para obtener una version más específica

Diseño

```

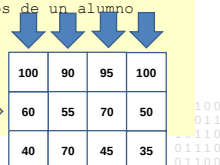
NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas():entero
cantAprobados():entero
cantAprobadosFila(f:entero):entero
cantAprobadosColumna(c:entero):entero
hayTodosDesaprobados():boolean
dosConsecutivosDes(f:entero):boolean
    
```

- **cantAprobados():entero**
Computa la cantidad de notas mayores al mínimo de aprobación.
- **cantAprobadosFila(f:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el alumno que corresponde a la fila f. Requiere f válida
- **cantAprobadosColumna(c:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el parcial que corresponde a la columna c. Requiere c válida.
- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Patrones de algoritmos – Contar

```

Algoritmo contar los parciales aprobados de un alumno
DE f
contador ← 0
para cada columna c
    si np,c > 60
        contador ← contador + 1
    
```



100	90	95	100
60	55	70	50
40	70	45	35

Recorrido de la fila 1 que mantiene las notas de un alumno

Diseño

```

NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas():entero
cantAprobados():entero
cantAprobadosFila(f:entero):entero
cantAprobadosColumna(c:entero):entero
hayTodosDesaprobados():boolean
dosConsecutivosDes(f:entero):boolean
    
```

- **cantAprobados():entero**
Computa la cantidad de notas mayores al mínimo de aprobación.
- **cantAprobadosFila(f:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el alumno que corresponde a la fila f. Requiere f válida
- **cantAprobadosColumna(c:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el parcial que corresponde a la columna c. Requiere c válida.
- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Patrones de algoritmos – Contar

Algoritmo contar los alumnos aprobados en un parcial

```
DE c
contador ← 0
para cada fila f
  si npf,c > 60
    contador ← contador + 1
```

100	90	95	100
60	55	70	50
40	70	45	35

Recorrido de la columna 3 que mantiene las notas de un parcial

Diseño

NotasParciales

```
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [][] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas(): entero
cantAprobados(): entero
cantAprobadosFila(f:entero): entero
cantAprobadosColumna(c:entero): entero
hayTodosDesaprobados(): boolean
dosConsecutivosDes(f:entero): boolean
```

- **cantAprobados():entero**
Computa la cantidad de notas mayores al mínimo de aprobación.
- **cantAprobadosFila(f:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el alumno que corresponde a la fila f. Requiere f válida
- **cantAprobadosColumna(c:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el parcial que corresponde a la columna c. Requiere c válida.
- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Recorrido

- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.

100	90	95	100
55	50	70	50
40	75	45	35

Retorna false

Recorrido

- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.

100	90	95	100
55	50	45	50
40	75	45	35

Retorna true

Recorridos no exhaustivos

Algoritmo decide si al menos un alumno desaprobó todos los parciales

```
para cada fila f y mientras no encuentre un alumno con
  todos los parciales desaprobados
  para cada columna c y mientras no encuentre un parcial
    aprobado
    si npf,c > 60
      encontró un aprobado
```

El algoritmo nos permite identificar las estructuras de control, hay todavía varios detalles a considerar antes de implementar en Java

Diseño

NotasParciales

```
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [][] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas(): entero
cantAprobados(): entero
cantAprobadosFila(f:entero): entero
cantAprobadosColumna(c:entero): entero
hayTodosDesaprobados(): boolean
dosConsecutivosDes(f:entero): boolean
Requiere que todas las notas estén
almacenadas cuando se ejecute una consulta
```

- **cantAprobados():entero**
Computa la cantidad de notas mayores al mínimo de aprobación.
- **cantAprobadosFila(f:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el alumno que corresponde a la fila f. Requiere f válida
- **cantAprobadosColumna(c:entero):entero**
Computa la cantidad de notas mayores al mínimo de aprobación para el parcial que corresponde a la columna c. Requiere c válida.
- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un alumno que desaprobó todos los parciales.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Tester dosConsecutivosDes

```

NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [][] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
<<Consultas>>
establecerNota(n,f,c:entero)
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas():entero
cantAprobados():entero
cantAprobadosFila(f:entero):entero
cantAprobadosColumna(c:entero):entero
hayTodosDesaprobados():boolean
dosConsecutivosDes(f:entero):boolean
Requiere que todas las notas estén
almacenadas cuando se ejecute una consulta
    
```

- ✓ La matriz tiene una sola columna, es decir hubo un solo parcial.
- ✓ Hubo dos parciales y ambos están desaprobados en la fila f
- ✓ Hubo dos parciales y ambos están aprobados en la fila f
- ✓ Hubo dos parciales y solo uno está aprobado
- ✓ Hubo más de dos parciales, los dos primeros desaprobados
- ✓ Hubo más de dos parciales, solo los dos últimos desaprobados
- ✓ Hubo más de dos parciales, dos desaprobados pero no son consecutivos.
- **dosConsecutivosDes(f:entero):boolean**
Retorna true si el alumno que corresponde a la fila f desaprobó dos parciales consecutivos.

Atención con la funcionalidad

```

NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [][] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero)
<<Consultas>>
obtenerNota(f,c:entero): entero
cantFilas(): entero
cantColumnas():entero
cantAprobados():entero
cantAprobadosFila(f:entero):entero
cantAprobadosColumna(c:entero):entero
hayTodosDesaprobados():boolean
dosConsecutivosDes(f:entero):boolean
Requiere que todas las notas estén
almacenadas cuando se ejecute una consulta
    
```

- **hayTodosDesaprobados():boolean**
Retorna true si hay al menos un parcial que fue desaprobado por todos los alumnos.

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

Atención con el contrato

```

NotasParciales
<<atributos de clase>>
minimo_aprobacion=60
<<atributos de instancia>>
np [][] entero
<<Constructores>>
NotasParciales(nFil,NCol: entero)
<<Comandos>>
establecerNota(n,f,c:entero):boolean
    
```

- **establecerNota(n, f,c:entero):boolean**
Si la posición es válida y la nota está en el rango 0-100, asigna n a la posición f,c y retorna true, sino retorna false.

```

01100
10011
10110
01110
01100
10011
10110
01110
    
```

Cambia la signature del comando **establecerNota** y el contrato.
El control de la posición es responsabilidad de la clase **NotasParciales**